C++
Complete Guide

Basics to Advanced — Beginner Edition

</> Code Verification | [G] Visual Diagrams
[>] Exercises | [*] Real World Examples
git Git Integration | [$] CMake Projects

Version 1.0 **March 6, 2026**

Claude by Anthropic

"C++ is a zero-overhead abstraction language" — Bjarne Stroustrup

Contents

List of Tables

Chapter 1

Introduction to C++

1.1 What is C++?

`C++` ek **general-purpose, compiled, statically-typed** programming language hai jo **Bjarne Stroustrup** ne 1979 mein Bell Labs mein develop ki. Yeh *C with Classes* ke naam se shuru hui aur aaj duniya ki sabse powerful languages mein se ek hai.

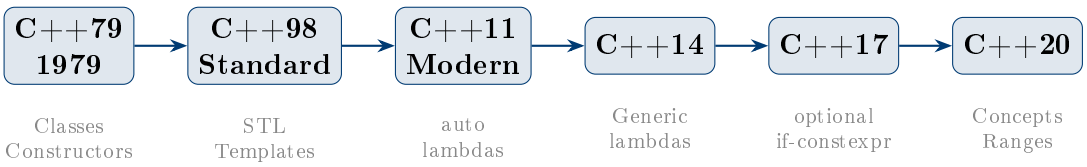
[*] Real World Connection

C++ kahan use hota hai?

- **Operating Systems:** Windows kernel, Linux drivers, macOS
- **Game Engines:** Unreal Engine, id Tech (Doom/Quake)
- **Browsers:** Google Chrome (V8 engine), Mozilla Firefox
- **Databases:** MySQL, MongoDB, ScyllaDB, Redis
- **Finance:** High-frequency trading (nanosecond latency)
- **Embedded:** Arduino, medical devices, car ECUs

GitHub: https://github.com/TheAlgorithms/C-Plus-Plus

1.2 C++ Evolution — Timeline



1.3 Pehla Program: Hello World

```cpp
// hello.cpp
// Compile: g++ -std=c++17 -Wall -o hello hello.cpp
// Run     : ./hello

#include <iostream>      // Input/Output stream library

int main() {             // Program entry point
    std::cout << "Hello, World!" << std::endl;
    // std::cout  = character output (to screen)
    // <<         = stream insertion operator
```

```
    // std::endl  = newline + flush buffer
    return 0;            // 0 = success
}
```

Listing 1.1: hello.cpp — Your first C++ program

[$] Compile Command

```
$ g++ -std=c++17 -Wall -o hello hello.cpp && ./hello
```

[ok] Expected Output

```
Hello, World!
```

[x] Common Error

Error: `'cout' was not declared in this scope`

Reason: `std::` prefix bhool gaye ya `iostream` include nahi kiya.
Fix: Ya `std::cout` likhein, ya file ke upar `using namespace std;` add karein.

[>] Try This Yourself

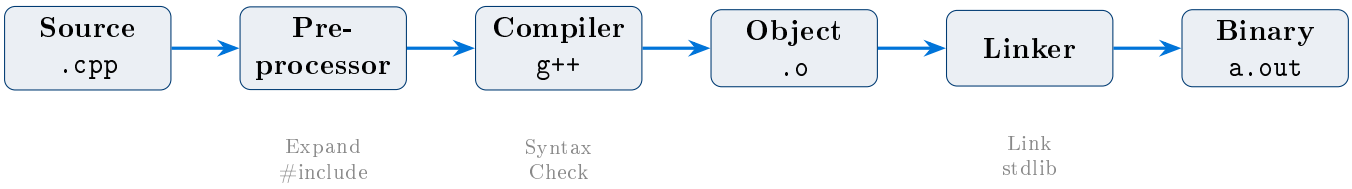
Exercise 1.1: Apna naam aur city print karo.

Exercise 1.2: Teen lines mein print karo: `Name: Arjun`, `Age: 20`, `City: Mumbai`

Solution:

```
#include <iostream>
int main() {
    std::cout << "Name: Arjun"  << "\n";
    std::cout << "Age: 20"      << "\n";
    std::cout << "City: Mumbai" << "\n";
    return 0;
}
```

1.4 Compilation Process — Step by Step



Chapter 2

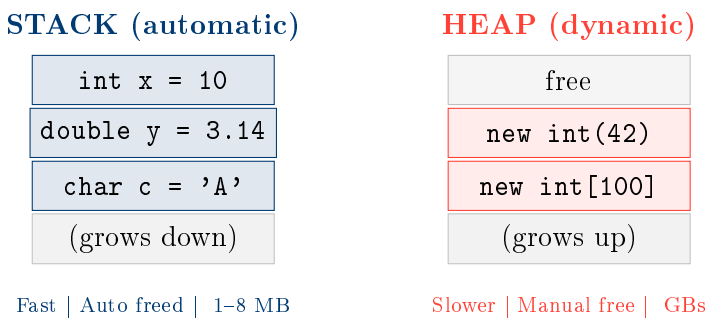
Variables, Data Types & Operators

2.1 Fundamental Data Types

Table 2.1: C++ Primitive Data Types

Type	Size	Range	Example
`int`	4 bytes	-2^{31} to $2^{31}-1$	`int age = 20;`
`long long`	8 bytes	-2^{63} to $2^{63}-1$	`long long n = 1e18;`
`float`	4 bytes	$\pm 3.4 \times 10^{38}$	`float pi = 3.14f;`
`double`	8 bytes	$\pm 1.7 \times 10^{308}$	`double e = 2.718;`
`char`	1 byte	0–255 (ASCII)	`char c = 'A';`
`bool`	1 byte	true / false	`bool ok = true;`
`string`	varies	any text	`string s = "Hi";`

2.2 Memory Layout — Stack vs Heap




```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Stack variables (automatic storage)
    int    age    = 20;
    double salary = 75000.50;
    char   grade  = 'A';
    bool   isPass = true;
    string name   = "Rahul";

```

```cpp
    // C++11: auto keyword - compiler deduces type
    auto score = 95;          // int
    auto gpa   = 3.8;         // double
    auto city  = "Delhi";     // const char*

    // Constants
    const double PI  = 3.14159265358979;
    constexpr int MAX = 100;  // compile-time constant

    cout << "Name   : " << name   << "\n"
         << "Age    : " << age    << "\n"
         << "Salary : " << salary << "\n"
         << "Grade  : " << grade  << "\n"
         << "Pass?  : " << boolalpha << isPass << "\n"
         << "Score  : " << score  << "\n"
         << "PI     : " << PI     << "\n";
    return 0;
}
```

Listing 2.1: variables_demo.cpp

[$] Compile Command

```
$ g++ -std=c++17 -Wall -o vars variables_demo.cpp && ./vars
```

[ok] Expected Output

```
Name :  Rahul
Age :  20
Salary :  75000.5
Grade :  A
Pass?  :  true
Score :  95
PI : 3.14159
```

2.3 Operators

```cpp
#include <iostream>
using namespace std;

int main() {
    // Arithmetic
    int a = 17, b = 5;
    cout << a + b  << "\n";   // 22  Addition
    cout << a - b  << "\n";   // 12  Subtraction
    cout << a * b  << "\n";   // 85  Multiplication
    cout << a / b  << "\n";   // 3   Integer division
       (truncates)
```

```
    cout << a % b  << "\n";    // 2   Modulo (remainder)

    // Comparison (returns bool)
    cout << (a > b)  << "\n"; // 1 (true)
    cout << (a == b) << "\n"; // 0 (false)
    cout << (a != b) << "\n"; // 1 (true)

    // Logical
    bool x = true, y = false;
    cout << (x && y) << "\n"; // 0 (AND)
    cout << (x || y) << "\n"; // 1 (OR)
    cout << (!x)     << "\n"; // 0 (NOT)

    // Bitwise
    int p = 0b1010, q = 0b1100;  // binary literals (C++14)
    cout << (p & q)  << "\n";    // 8  AND
    cout << (p | q)  << "\n";    // 14 OR
    cout << (p ^ q)  << "\n";    // 6  XOR
    cout << (p << 1) << "\n";    // 20 left shift = multiply by
       2
    cout << (p >> 1) << "\n";    // 5  right shift = divide by 2

    // Ternary operator
    int max_val = (a > b) ? a : b;
    cout << "Max: " << max_val << "\n"; // 17

    // Assignment shortcuts
    int c = 10;
    c += 5;  c -= 3;  c *= 2;  c /= 4;  c %= 4;
    cout << "c = " << c << "\n"; // 2

    return 0;
}
```

Listing 2.2: operators.cpp

[?] Spot the Bug

Yeh code kya galat print karega?

```
int a = 5, b = 2;
double result = a / b;
cout << result;   // kya print hoga?
```

Bug: `a/b` integer division karta hai = 2, phir double mein convert hota hai = 2.0 (not 2.5!)
Fix: `double result = (double)a / b;`
ya: `double result = a / 2.0;`

Chapter 3

Control Flow

3.1 if / else if / else

```cpp
#include <iostream>
using namespace std;

int main() {
    int marks;
    cout << "Enter marks (0-100): ";
    cin >> marks;

    if (marks >= 90) {
        cout << "Grade: A+ (Excellent!)\n";
    } else if (marks >= 80) {
        cout << "Grade: A  (Very Good)\n";
    } else if (marks >= 70) {
        cout << "Grade: B  (Good)\n";
    } else if (marks >= 60) {
        cout << "Grade: C  (Average)\n";
    } else if (marks >= 40) {
        cout << "Grade: D  (Pass)\n";
    } else {
        cout << "Grade: F  (Fail)\n";
    }
    return 0;
}
```

Listing 3.1: grades.cpp

[$] Compile Command

```
$ g++ -std=c++17 -o grades grades.cpp && echo "85" | ./grades
```

[ok] Expected Output

```
Enter marks (0-100):  Grade:  A (Very Good)
```

3.2 Switch Statement

```cpp
#include <iostream>
```

```cpp
using namespace std;

int main() {
    double a, b; char op;
    cout << "Enter: num op num (e.g. 10 + 5): ";
    cin >> a >> op >> b;

    switch (op) {
        case '+': cout << a + b;     break;
        case '-': cout << a - b;     break;
        case '*': cout << a * b;     break;
        case '/':
            if (b == 0) { cout << "Error: Div by zero!"; break; }
            cout << a / b;
            break;
        default:
            cout << "Unknown operator!";
    }
    cout << "\n";
    return 0;
}
```

Listing 3.2: calculator.cpp

> [!] Warning
>
> **break bhoolna = Fall-through bug!**
> Agar `break` nahi likha, control next case mein *fall-through* ho jaata hai — bahut common beginner mistake!

3.3 Loops

```cpp
#include <iostream>
using namespace std;

int main() {
    // Classic for loop
    for (int i = 1; i <= 5; i++) {
        cout << i << " ";    // 1 2 3 4 5
    }
    cout << "\n";

    // Range-based for (C++11) - most modern, preferred
    int arr[] = {10, 20, 30, 40, 50};
    for (int x : arr) {
        cout << x << " ";    // 10 20 30 40 50
```

```
    }
    cout << "\n";

    // while loop
    int n = 1;
    while (n <= 5) { cout << n << " "; n++; }
    cout << "\n"; // 1 2 3 4 5

    // do-while: runs at least ONCE
    int input;
    do {
        cout << "Enter positive number: ";
        cin >> input;
    } while (input <= 0);

    // Nested loops - multiplication table
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= 5; j++)
            cout << i*j << "\t";
        cout << "\n";
    }

    // break and continue
    for (int i = 1; i <= 10; i++) {
        if (i == 6) break;          // stop at 6
        if (i % 2 == 0) continue;   // skip even
        cout << i << " ";           // 1 3 5
    }
    cout << "\n";

    return 0;
}
```

Listing 3.3: loops.cpp

[>] Try This Yourself

Exercise 3.1 — Pattern Printing:

```
*
**
***
****
*****
```

Solution:

```
for (int i = 1; i <= 5; i++) {
    for (int j = 0; j < i; j++) cout << "*";
    cout << "\n";
```

```
}
```

Exercise 3.2 — Sum of digits of a number:
Hint: Use `n % 10` to get last digit, `n / 10` to remove it.

```
int n = 12345, sum = 0;
while (n > 0) { sum += n % 10; n /= 10; }
cout << "Sum = " << sum; // 15
```

Chapter 4

Functions

4.1 Function Basics

```cpp
#include <iostream>
#include <cmath>
using namespace std;

// Function declarations (prototypes)
double circleArea(double radius);
int    factorial(int n);
bool   isPrime(int n);

int main() {
    cout << "Area(r=5)  : " << circleArea(5) << "\n";
    cout << "5!         : " << factorial(5)  << "\n";
    cout << "17 prime?  : " << boolalpha << isPrime(17) << "\n";
    return 0;
}

double circleArea(double r) {
    return 3.14159265 * r * r;
}

int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);  // Recursion!
}

bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
```

Listing 4.1: functions_intro.cpp

[$] Compile Command

```
$ g++ -std=c++17 -Wall -o functions functions_intro.cpp &&
./functions
```

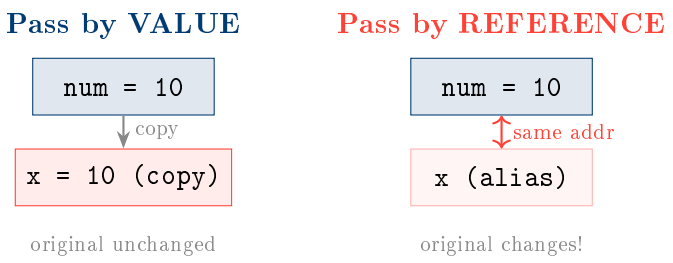
[ok] Expected Output

```
Area(r=5) :  78.5398
5!  :  120
17 prime?  :  true
```

4.2 Pass by Value vs Pass by Reference

```cpp
#include <iostream>
using namespace std;

// Pass by VALUE: copy banata hai, original safe
void doubleValue(int x) {
    x *= 2;    // local copy
    cout << "Inside: " << x;
}

// Pass by REFERENCE: original modify hota hai
void doubleRef(int& x) {    // & = reference
    x *= 2;                 // original change
}

// Pass by CONST REFERENCE: efficient, no copy, no modify
void printStr(const string& s) {
    cout << s;
}

int main() {
    int num = 10;

    doubleValue(num);
    cout << " | After: " << num << "\n";    // Inside: 20 | After: 10

    doubleRef(num);
    cout << "After ref: " << num << "\n";  // After ref: 20

    string msg = "Hello from reference!";
    printStr(msg);
    cout << "\n";

    return 0;
}
```

Listing 4.2: pass_by.cpp



4.3 Function Overloading

```
#include <iostream>
#include <string>
using namespace std;

// Same name, different parameters - compiler picks the right
    one
int    add(int a, int b)        { return a + b; }
double add(double a, double b)  { return a + b; }
string add(string a, string b)  { return a + b; }
int    add(int a, int b, int c) { return a + b + c; }

int main() {
    cout << add(3, 4)              << "\n"; // 7
    cout << add(3.5, 4.2)          << "\n"; // 7.7
    cout << add(string("Hello"),
                string("World")) << "\n"; // HelloWorld
    cout << add(1, 2, 3)           << "\n"; // 6
    return 0;
}
```

Listing 4.3: overloading.cpp

4.4 Lambda Functions (C++11)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    // Syntax: [capture](params) -> return_type { body }
    auto greet = [](string name) {
        cout << "Hello, " << name << "!\n";
    };
    greet("Anjali");  // Hello, Anjali!

```

```
    // Capture outer variable by value
    int threshold = 50;
    auto isAbove = [threshold](int x) { return x > threshold; };
    cout << isAbove(60) << "\n"; // 1

    // Lambda in sort (descending order)
    vector<int> nums = {5, 2, 8, 1, 9, 3};
    sort(nums.begin(), nums.end(),
         [](int a, int b) { return a > b; });
    for (int n : nums) cout << n << " "; // 9 8 5 3 2 1
    cout << "\n";

    return 0;
}
```

Listing 4.4: lambdas.cpp

[*] Real World Connection

Lambdas in Industry:
Qt (C++ GUI framework) mein button click handler lambda se likha jaata hai:

```
connect(button, &QPushButton::clicked, [&]() {
    label->setText("Button Clicked!");
});
```

Google's `abseil` library mein bhi extensively use hote hain.
GitHub: https://github.com/abseil/abseil-cpp

Chapter 5

Arrays, Strings & Vectors

5.1 Arrays

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    // 1D Array declaration + initialization
    int scores[5] = {85, 92, 78, 95, 88};
    int n = 5;

    // Traversal
    for (int i = 0; i < n; i++)
        cout << scores[i] << " ";    // 85 92 78 95 88
    cout << "\n";

    // Sort (ascending)
    sort(scores, scores + n);
    for (int x : scores) cout << x << " "; // 78 85 88 92 95
    cout << "\n";

    // Min, Max, Sum
    int minVal = *min_element(scores, scores + n);
    int maxVal = *max_element(scores, scores + n);
    int sum = 0;
    for (int x : scores) sum += x;
    cout << "Min:" << minVal << " Max:" << maxVal
         << " Avg:" << (double)sum/n << "\n";

    // 2D Array (matrix)
    int mat[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++)
            cout << mat[i][j] << " ";
        cout << "\n";
    }
    return 0;
}
```

Listing 5.1: arrays.cpp

[$] Compile Command

```
$ g++ -std=c++17 -o arrays arrays.cpp && ./arrays
```

[ok] Expected Output

```
85 92 78 95 88
78 85 88 92 95
Min:78 Max:95 Avg:87.6
1 2 3
4 5 6
7 8 9
```

5.2 Strings

```cpp
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

int main() {
    string s = "Hello, C++ World!";

    cout << s.length()       << "\n"; // 17
    cout << s.substr(7, 3)   << "\n"; // C++
    cout << s.find("World") << "\n"; // 11
    cout << s[0]             << "\n"; // H

    // Modify
    s.replace(7, 3, "Modern");  // Hello, Modern World!
    s += " (2024)";               // append

    // Reverse
    string word = "racecar";
    string rev = word;
    reverse(rev.begin(), rev.end());
    cout << word << (word == rev ? " IS" : " is NOT")
         << " a palindrome\n";

    // Split by delimiter (manual approach)
    string csv = "apple,banana,cherry";
    size_t pos;
    while ((pos = csv.find(',')) != string::npos) {
        cout << csv.substr(0, pos) << "\n";
        csv.erase(0, pos + 1);
    }
    cout << csv << "\n"; // cherry

```

```
    return 0;
}
```

Listing 5.2: strings.cpp

5.3 Vectors — Dynamic Arrays

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>   // accumulate
using namespace std;

int main() {
    vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

    v.push_back(7);            // add at end
    v.pop_back();              // remove from end
    v.insert(v.begin(), 0);    // insert at front
    v.erase(v.begin() + 3);    // erase at index 3

    cout << "Size: "     << v.size()     << "\n";
    cout << "Capacity: " << v.capacity() << "\n";

    sort(v.begin(), v.end());
    cout << "Sorted: ";
    for (int x : v) cout << x << " ";
    cout << "\n";

    bool found = binary_search(v.begin(), v.end(), 5);
    cout << "5 found? " << boolalpha << found << "\n";

    int sum = accumulate(v.begin(), v.end(), 0);
    cout << "Sum: " << sum << "\n";

    // 2D Vector (dynamic matrix)
    int rows = 3, cols = 4;
    vector<vector<int>> mat(rows, vector<int>(cols, 0));
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            mat[i][j] = i * cols + j;

    for (auto& row : mat) {
        for (int x : row) cout << x << "\t";
        cout << "\n";
    }
    return 0;
}
```

Listing 5.3: vectors.cpp

Chapter 6

Pointers & References

6.1 What is a Pointer?

Ek pointer ek variable hai jo kisi dusre variable ka **memory address** store karta hai.

```cpp
#include <iostream>
using namespace std;

int main() {
    int x = 42;
    int* ptr = &x;    // ptr stores address of x

    cout << "Value of x     : " << x    << "\n";  // 42
    cout << "Address of x   : " << &x   << "\n";  // 0x7fff...
    cout << "ptr holds addr : " << ptr  << "\n";  // same
    cout << "Value at ptr   : " << *ptr << "\n";  // 42

    // Modify through pointer
    *ptr = 100;
    cout << "x after *ptr=100: " << x << "\n";    // 100

    // Pointer arithmetic
    int arr[] = {10, 20, 30, 40, 50};
    int* p = arr;                // points to arr[0]
    cout << *(p + 2) << "\n";  // 30 (arr[2])
    cout << p[3]     << "\n";  // 40 (arr[3])

    // Null pointer (safe practice)
    int* safe = nullptr;
    if (safe == nullptr)
        cout << "Pointer is null --- safe!\n";

    return 0;
}
```

Listing 6.1: pointers_intro.cpp

6.2 Dynamic Memory Allocation

```cpp
#include <iostream>
using namespace std;

int main() {
    // Single int on heap
    int* p = new int(10);
    cout << *p << "\n";        // 10
    delete p;                  // MUST free! else memory leak
    p = nullptr;               // good practice

    // Array on heap
    int n = 5;
    int* arr = new int[n];
    for (int i = 0; i < n; i++) arr[i] = i * 10;
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << "\n";
    delete[] arr;              // [] for arrays!
    arr = nullptr;

    return 0;
}
```

Listing 6.2: dynamic_memory.cpp

[$] Compile Command

```
$ g++ -std=c++17 -o dynmem dynamic_memory.cpp && ./dynmem
```

[ok] Expected Output

```
10
0 10 20 30 40
```

[!] Warning

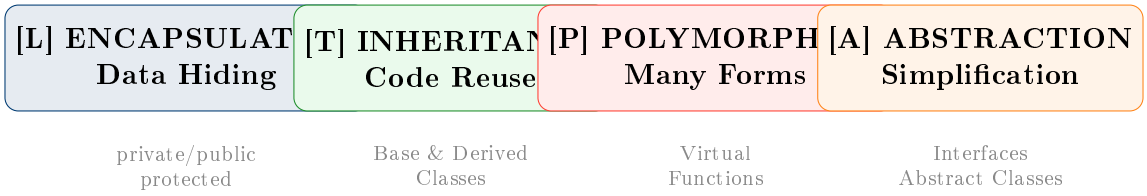
Memory Leak kya hota hai?
Agar `new` ke baad `delete` na karo, woh memory program ke exit hone tak locked rehti hai. Bade programs mein yeh GBs tak ja sakta hai!

Detect karo: `valgrind -leak-check=full ./program`
Modern fix: Smart pointers use karo (Chapter 9)!

Chapter 7

Object-Oriented Programming (OOP)

7.1 OOP ke 4 Pillars



7.2 Classes and Objects

```
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;

class BankAccount {
private:                          // Encapsulation: hidden from outside
    string owner;
    double balance;
    string accountNo;
    static int totalAccounts;     // shared across ALL objects

public:
    // Constructor
    BankAccount(string name, string accNo, double init = 0.0)
        : owner(name), accountNo(accNo), balance(init) {
        if (init < 0)
            throw invalid_argument("Balance cannot be negative");
        totalAccounts++;
        cout << "Account created: " << owner << "\n";
    }

    // Destructor
    ~BankAccount() {
        totalAccounts--;
    }

```

```
    void deposit(double amount) {
        if (amount <= 0) throw invalid_argument("Invalid
           amount");
        balance += amount;
        cout << "Deposited: " << amount
             << " | Balance: " << balance << "\n";
    }

    void withdraw(double amount) {
        if (amount > balance)
            throw runtime_error("Insufficient funds!");
        balance -= amount;
        cout << "Withdrawn: " << amount
             << " | Balance: " << balance << "\n";
    }

    // Getters
    double getBalance() const { return balance; }
    string getOwner()   const { return owner;   }

    static int getTotalAccounts() { return totalAccounts; }

    friend ostream& operator<<(ostream& os, const BankAccount&
       a) {
        os << "Account[" << a.accountNo << "] " << a.owner
           << " Rs." << a.balance;
        return os;
    }
};

int BankAccount::totalAccounts = 0; // static member init

int main() {
    BankAccount acc1("Rahul", "ACC001", 5000.0);
    BankAccount acc2("Priya", "ACC002", 2000.0);

    cout << "Total accounts: "
         << BankAccount::getTotalAccounts() << "\n\n";

    acc1.deposit(1500.0);
    acc1.withdraw(800.0);

    try {
        acc2.withdraw(5000.0);  // will throw exception
    } catch (const runtime_error& e) {
        cout << "Error: " << e.what() << "\n";
    }

    cout << "\n" << acc1 << "\n" << acc2 << "\n";
```

```
    return 0;
}
```

Listing 7.1: BankAccount.cpp

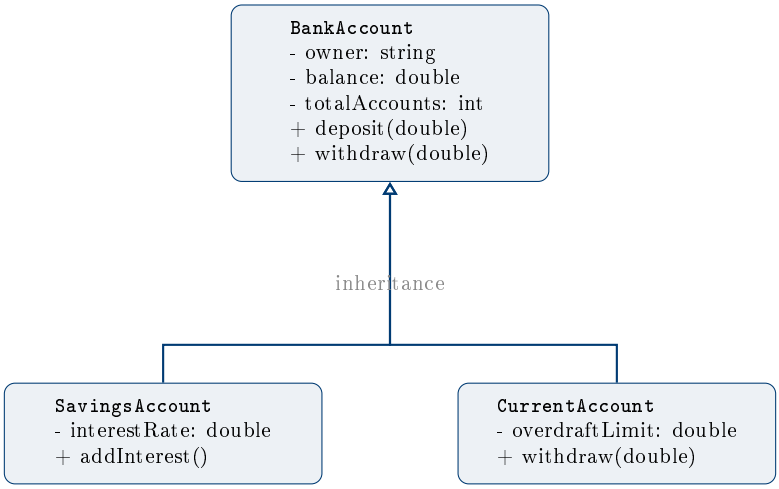
[$] Compile Command

```
$ g++ -std=c++17 -Wall -o bank BankAccount.cpp && ./bank
```

[ok] Expected Output

```
Account created:  Rahul
Account created:  Priya
Total accounts:  2
Deposited:  1500 | Balance:  6500
Withdrawn:  800 | Balance:  5700
Error:  Insufficient funds!
```

7.3 Class Hierarchy Diagram



7.4 Inheritance & Polymorphism

```
#include <iostream>
using namespace std;

class Animal {
protected:
    string name;
public:
    Animal(string n) : name(n) {}
    virtual void speak() {            // virtual = can be
        overridden
```

```
        cout << name << " makes a sound\n";
    }
    virtual ~Animal() {}              // virtual destructor (important!)
};

class Dog : public Animal {
public:
    Dog(string n) : Animal(n) {}
    void speak() override {           // override keyword (C++11)
        cout << name << " says: Woof! Woof!\n";
    }
    void fetch() { cout << name << " fetches the ball!\n"; }
};

class Cat : public Animal {
public:
    Cat(string n) : Animal(n) {}
    void speak() override {
        cout << name << " says: Meow~\n";
    }
};

class Parrot : public Animal {
    string phrase;
public:
    Parrot(string n, string p) : Animal(n), phrase(p) {}
    void speak() override {
        cout << name << " says: " << phrase << "\n";
    }
};

// Polymorphism: same call, different behaviour at runtime
void makeItSpeak(Animal* animal) {
    animal->speak();
}

int main() {
    Dog    d("Bruno");
    Cat    c("Whiskers");
    Parrot p("Mitthu", "Hello Mitthu!");

    Animal* zoo[] = {&d, &c, &p};
    for (Animal* a : zoo)
        makeItSpeak(a);   // calls correct version!

    d.fetch();
    return 0;
}
```

Listing 7.2: inheritance.cpp

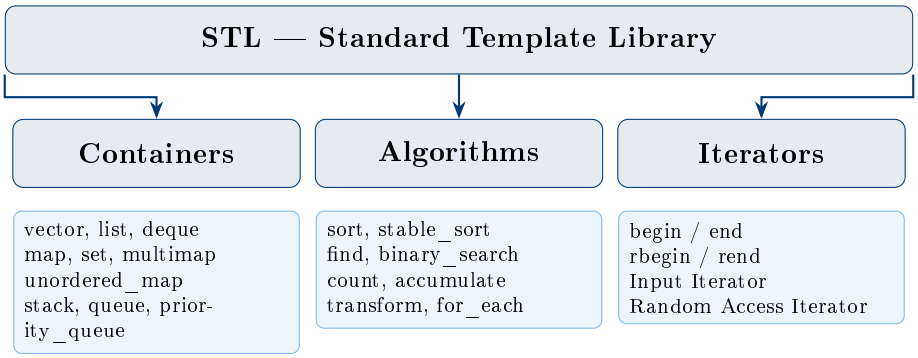
[ok] Expected Output

```
Bruno says:  Woof!  Woof!
Whiskers says:  Meow~
Mitthu says:  Hello Mitthu!
Bruno fetches the ball!
```

Chapter 8

STL — Standard Template Library

8.1 STL Overview



8.2 Essential Containers

```
#include <iostream>
#include <vector>
#include <map>
#include <set>
#include <unordered_map>
#include <queue>
#include <stack>
using namespace std;

int main() {
    // vector (dynamic array)
    vector<int> v = {3,1,4,1,5,9};
    v.push_back(2);
    sort(v.begin(), v.end());

    // map (sorted key-value pairs, O(log n))
    map<string, int> wordCount;
    wordCount["hello"]++;
    wordCount["world"]++;
    wordCount["hello"]++;
    for (auto& [word, cnt] : wordCount)  // C++17 structured bindings
        cout << word << ": " << cnt << "\n";

    // unordered_map (hash map, O(1) average)
```

```cpp
    unordered_map<string, int> scores;
    scores["Amit"]  = 95;
    scores["Priya"] = 88;
    cout << "Amit: " << scores["Amit"] << "\n";

    // set (unique sorted elements)
    set<int> s = {5,3,1,4,2,3,1}; // duplicates removed
    for (int x : s) cout << x << " "; // 1 2 3 4 5
    cout << "\n";

    // stack (LIFO)
    stack<int> stk;
    stk.push(1); stk.push(2); stk.push(3);
    while (!stk.empty()) {
        cout << stk.top() << " "; stk.pop();
    }
    cout << "\n"; // 3 2 1

    // queue (FIFO)
    queue<string> q;
    q.push("First"); q.push("Second"); q.push("Third");
    while (!q.empty()) {
        cout << q.front() << " "; q.pop();
    }
    cout << "\n"; // First Second Third

    // priority_queue (max-heap by default)
    priority_queue<int> pq;
    pq.push(5); pq.push(1); pq.push(9); pq.push(3);
    while (!pq.empty()) {
        cout << pq.top() << " "; pq.pop();
    }
    cout << "\n"; // 9 5 3 1

    return 0;
}
```

Listing 8.1: stl_containers.cpp

[$] Compile Command

```
$ g++ -std=c++17 -Wall -o stl stl_containers.cpp && ./stl
```

8.3 STL Algorithms

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
```

```cpp
#include <numeric>
using namespace std;

int main() {
    vector<int> v(10);
    iota(v.begin(), v.end(), 1);  // fill: 1 2 3 ... 10

    // accumulate (sum)
    int sum = accumulate(v.begin(), v.end(), 0);
    cout << "Sum: " << sum << "\n"; // 55

    // count_if
    int evens = count_if(v.begin(), v.end(),
                         [](int x){ return x % 2 == 0; });
    cout << "Evens: " << evens << "\n"; // 5

    // transform (square each element)
    vector<int> sq(v.size());
    transform(v.begin(), v.end(), sq.begin(),
              [](int x){ return x * x; });
    for (int x : sq) cout << x << " ";
    cout << "\n"; // 1 4 9 16 25 36 49 64 81 100

    // find_if
    auto it = find_if(v.begin(), v.end(),
                      [](int x){ return x > 7; });
    if (it != v.end())
        cout << "First > 7: " << *it << "\n"; // 8

    // all_of, any_of, none_of
    cout << all_of(v.begin(),v.end(),[](int x){return x>0;})
         << "\n"; // 1

    // reverse
    reverse(v.begin(), v.end());
    for (int x : v) cout << x << " "; // 10 9 8 ... 1
    cout << "\n";

    return 0;
}
```

Listing 8.2: stl_algorithms.cpp

Chapter 9

Templates & Generic Programming

9.1 Function Templates

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

// Function template: works for any comparable type
template <typename T>
T myMax(T a, T b) { return (a > b) ? a : b; }

// Multiple template parameters
template <typename T, typename U>
void printPair(T a, U b) {
    cout << "(" << a << ", " << b << ")\n";
}

// Class template
template <typename T>
class Stack {
    vector<T> data;
public:
    void push(const T& val) { data.push_back(val); }
    void pop()              { data.pop_back(); }
    T    top()  const       { return data.back(); }
    bool empty() const      { return data.empty(); }
    int  size()  const      { return (int)data.size(); }
};

int main() {
    cout << myMax(3, 7)            << "\n"; // 7
    cout << myMax(3.14, 2.71)      << "\n"; // 3.14
    cout << myMax(string("cat"),
                  string("dog"))   << "\n"; // dog

    printPair(1, "hello");           // (1, hello)
    printPair(3.14, true);           // (3.14, 1)

    Stack<int> intStack;
    intStack.push(10); intStack.push(20); intStack.push(30);
    while (!intStack.empty()) {
```

```
        cout << intStack.top() << " "; intStack.pop();
    }
    cout << "\n"; // 30 20 10

    Stack<string> strStack;
    strStack.push("Delhi"); strStack.push("Mumbai");
    cout << strStack.top() << "\n"; // Mumbai

    return 0;
}
```

Listing 9.1: templates.cpp

[$] Compile Command

```
$ g++ -std=c++17 -Wall -o tmpl templates.cpp && ./tmpl
```

[ok] Expected Output

```
7
3.14
dog
(1, hello)
(3.14, 1)
30 20 10
Mumbai
```

Chapter 10

Modern C++ (C++11/14/17)

10.1 Smart Pointers

```
#include <iostream>
#include <memory>
#include <string>
using namespace std;

class Resource {
    string name;
public:
    Resource(string n) : name(n) {
        cout << "Created: " << name << "\n";
    }
    ~Resource() {
        cout << "Destroyed: " << name << "\n";
    }
    void use() { cout << "Using: " << name << "\n"; }
};

int main() {
    // unique_ptr: single owner, auto-deleted when out of scope
    {
        auto up = make_unique<Resource>("FileHandle");
        up->use();
        auto up2 = move(up);   // transfer ownership
        up2->use();
        // up2 destroyed here automatically!
    }

    cout << "\n--- shared_ptr ---\n";
    {
        auto sp1 = make_shared<Resource>("Database");
        auto sp2 = sp1;   // shared ownership
        cout << "ref count: " << sp1.use_count() << "\n"; // 2
    }
    // destroyed when LAST shared_ptr goes out of scope

    cout << "\n--- nullptr check ---\n";
    unique_ptr<int> null_p;
    if (!null_p) cout << "Pointer is null (safe)\n";

```

```
    return 0;
}
```

Listing 10.1: smart_pointers.cpp

[$] Compile Command

```
$ g++ -std=c++17 -Wall -o smartptr smart_pointers.cpp && ./smartptr
```

[i] Note

Smart Pointer Rule:

- `unique_ptr` — Default choice. Single owner.
- `shared_ptr` — Multiple owners needed.
- `weak_ptr` — Break circular references.
- Modern C++ mein **never** use raw `new/delete`!

10.2 Move Semantics

```
#include <iostream>
#include <vector>
using namespace std;

class BigData {
    vector<int> data;
public:
    BigData(int size) : data(size, 42) {
        cout << "Constructed (n=" << size << ")\n";
    }
    BigData(const BigData& o) : data(o.data) {
        cout << "COPY - expensive!\n";
    }
    BigData(BigData&& o) noexcept : data(move(o.data)) {
        cout << "MOVE - cheap!\n";  // steals the data
    }
    size_t size() const { return data.size(); }
};

int main() {
    BigData a(1000000);        // Constructed

    BigData b = a;             // COPY (slow: copies 1M ints)
    BigData c = move(a);       // MOVE (fast: just pointer swap)

    cout << "b: " << b.size() << "\n"; // 1000000
    cout << "c: " << c.size() << "\n"; // 1000000
    cout << "a: " << a.size() << "\n"; // 0 (moved-from, empty)
```

```

    return 0;
}
```

Listing 10.2: move_semantics.cpp

10.3 C++17 Features

```
#include <iostream>
#include <optional>
#include <map>
#include <tuple>
using namespace std;

// optional: function "maybe" returns a value
optional<int> safeDivide(int a, int b) {
    if (b == 0) return nullopt;
    return a / b;
}

tuple<string, int, double> getEmployee() {
    return {"Vikram", 30, 85000.0};
}

int main() {
    // optional
    auto r1 = safeDivide(10, 2);
    if (r1) cout << "10/2 = " << *r1 << "\n";      // 5

    auto r2 = safeDivide(10, 0);
    cout << "10/0 = " << r2.value_or(-1) << "\n"; // -1

    // Structured bindings (C++17)
    auto [name, age, salary] = getEmployee();
    cout << name << " | " << age << " | " << salary << "\n";

    // Map iteration with structured bindings
    map<string,int> inv = {{"apple",5},{"banana",3}};
    for (auto& [item, qty] : inv)
        cout << item << ": " << qty << "\n";

    // if-init statement (C++17)
    if (auto it = inv.find("apple"); it != inv.end())
        cout << "Found apple: " << it->second << "\n";

    return 0;
}
```

Listing 10.3: cpp17_features.cpp

[$] Compile Command

```
$ g++ -std=c++17 -Wall -o cpp17 cpp17_features.cpp && ./cpp17
```

[ok] Expected Output

```
10/2 = 5
10/0 = -1
Vikram | 30 | 85000
apple:  5
banana:  3
Found apple:  5
```

Chapter 11

Complete Project: Student Management System

11.1 Project Structure

[D] Project Directory

```
StudentMS/
|-- CMakeLists.txt
|-- README.md
|-- .gitignore
|-- src/
|   |-- main.cpp
|   |-- Student.h
|   |-- Student.cpp
|-- tests/
|   |-- test_student.cpp
|-- data/
    |-- students.csv
```

11.2 CMakeLists.txt

```cmake
cmake_minimum_required(VERSION 3.16)
project(StudentMS VERSION 1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

# Compiler warnings
add_compile_options(-Wall -Wextra -Wpedantic)

# Debug / Release flags
if(CMAKE_BUILD_TYPE STREQUAL "Debug")
    add_compile_options(-g -O0 -fsanitize=address)
    add_link_options(-fsanitize=address)
endif()

# Main executable
```

```
add_executable(studentms
    src/main.cpp
    src/Student.cpp
)
target_include_directories(studentms PRIVATE src)

# Test executable
add_executable(run_tests
    tests/test_student.cpp
    src/Student.cpp
)
target_include_directories(run_tests PRIVATE src)
```

Listing 11.1: CMakeLists.txt

11.3 Student.h

```
#pragma once
#include <string>
#include <vector>
#include <iostream>

class Student {
private:
    int         id;
    std::string name;
    std::string email;
    int         age;
    double      gpa;
    std::vector<std::string> courses;

public:
    Student(int id, std::string name, std::string email,
            int age, double gpa = 0.0);

    int         getId()    const;
    std::string getName()  const;
    std::string getEmail() const;
    int         getAge()   const;
    double      getGpa()   const;

    void setGpa(double gpa);
    void enrollCourse(const std::string& course);
    void dropCourse(const std::string& course);
    bool isEnrolled(const std::string& course) const;
    std::string toCSV() const;

    bool operator==(const Student& other) const;
```

```
    bool operator<(const Student& other)  const;

    friend std::ostream& operator<<(std::ostream& os,
                                    const Student& s);
};
```

Listing 11.2: src/Student.h

11.4 Student.cpp

```
#include "Student.h"
#include <stdexcept>
#include <algorithm>
#include <sstream>
using namespace std;

Student::Student(int id, string name, string email,
                 int age, double gpa)
    : id(id), name(name), email(email), age(age), gpa(gpa) {
    if (gpa < 0.0 || gpa > 10.0)
        throw invalid_argument("GPA must be 0--10");
    if (age < 15 || age > 60)
        throw invalid_argument("Invalid age");
}

int    Student::getId()    const { return id; }
string Student::getName()  const { return name; }
string Student::getEmail() const { return email; }
int    Student::getAge()   const { return age; }
double Student::getGpa()   const { return gpa; }

void Student::setGpa(double g) {
    if (g < 0.0 || g > 10.0)
        throw invalid_argument("GPA must be 0-10");
    gpa = g;
}

void Student::enrollCourse(const string& course) {
    if (!isEnrolled(course)) courses.push_back(course);
}

void Student::dropCourse(const string& course) {
    courses.erase(
        remove(courses.begin(), courses.end(), course),
        courses.end());
}

bool Student::isEnrolled(const string& course) const {
```

```
    return find(courses.begin(), courses.end(), course)
            != courses.end();
}

string Student::toCSV() const {
    stringstream ss;
    ss << id << "," << name << "," << email << ","
       << age << "," << gpa;
    return ss.str();
}

bool Student::operator==(const Student& o) const {
    return id == o.id;
}
bool Student::operator<(const Student& o) const {
    return name < o.name;
}

ostream& operator<<(ostream& os, const Student& s) {
    os << "[" << s.id << "] " << s.name
       << " | GPA: " << s.gpa
       << " | Courses: " << s.courses.size();
    return os;
}
```

Listing 11.3: src/Student.cpp

11.5 main.cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <fstream>
#include "Student.h"
using namespace std;

vector<Student> db;
int nextId = 1;

void addStudent() {
    string name, email; int age; double gpa;
    cout << "Name  : "; getline(cin >> ws, name);
    cout << "Email : "; cin >> email;
    cout << "Age   : "; cin >> age;
    cout << "GPA   : "; cin >> gpa;
    try {
        db.emplace_back(nextId++, name, email, age, gpa);
        cout << "Student added!\n";
```

```
    } catch (exception& e) {
        cout << "Error: " << e.what() << "\n";
    }
}

void listStudents() {
    if (db.empty()) { cout << "No students.\n"; return; }
    cout << "\n=== All Students ===\n";
    for (const auto& s : db) cout << s << "\n";
}

void searchByName() {
    string q; cout << "Name to search: "; cin >> q;
    bool found = false;
    for (const auto& s : db)
        if (s.getName().find(q) != string::npos) {
            cout << s << "\n"; found = true;
        }
    if (!found) cout << "Not found.\n";
}

void saveToCSV() {
    ofstream f("data/students.csv");
    f << "ID,Name,Email,Age,GPA\n";
    for (const auto& s : db) f << s.toCSV() << "\n";
    cout << "Saved to data/students.csv\n";
}

int main() {
    int choice;
    do {
        cout << "\n1.Add  2.List  3.Search  4.Save  0.Exit\n"
             << "Choice: ";
        cin >> choice;
        switch (choice) {
            case 1: addStudent();   break;
            case 2: listStudents(); break;
            case 3: searchByName(); break;
            case 4: saveToCSV();    break;
            case 0: cout << "Bye!\n"; break;
        }
    } while (choice != 0);
    return 0;
}
```

Listing 11.4: src/main.cpp

11.6 Build and Run

[$] Compile Command

```
mkdir -p StudentMS/build && cd StudentMS/build
cmake .. -DCMAKE_BUILD_TYPE=Debug
make -j4
./studentms
```

Chapter 12

Git & Version Control

12.1 .gitignore for C++

```
# Build output
build/
cmake-build-*/
*.o
*.a
*.so
*.exe
*.out

# IDE files
.vscode/
.idea/
*.swp
.DS_Store

# Compiled binaries
studentms
run_tests

# CMake
CMakeCache.txt
CMakeFiles/
Makefile
```

Listing 12.1: .gitignore

12.2 Git Workflow

```
# Initial setup
git init
git add .gitignore
git commit -m "chore: add .gitignore for C++ project"

# Feature branch workflow
git checkout -b feature/student-search

# After coding...
```

```
git add src/main.cpp src/Student.cpp
git commit -m "feat(search): add search by name

- Partial match supported
- Handles no-result gracefully
- O(n) linear scan"

# Merge back
git checkout main
git merge feature/student-search --no-ff
git tag v1.0.0 -m "Release: initial working version"
git push origin main --tags
```

Listing 12.2: Git commands for C++ project

12.2.1 Commit Message Format

[i] Note

Conventional Commits Standard:

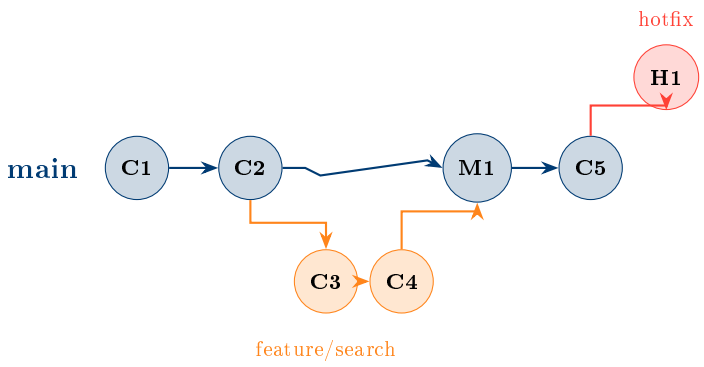
```
<type>(<scope>): <short description>

[optional body]
[optional footer]
```

Types: `feat` | `fix` | `docs` | `refactor` | `test` | `chore` | `perf`

Examples:

```
feat(student): add GPA validation
fix(db): prevent duplicate student IDs
refactor(main): extract menu into function
test: add unit tests for Student class
```

12.2.2 Branching Strategy



Chapter 13

Debugging & Best Practices

13.1 Common Bugs

[?] Spot the Bug

Bug 1: Off-by-one

```
int arr[5] = {1,2,3,4,5};
for (int i = 0; i <= 5; i++)  // BUG: i<=5 reads arr[5]!
    cout << arr[i];
```

Fix: Use `i < 5` (strict less than)

[?] Spot the Bug

Bug 2: Use-after-delete

```
int* p = new int(42);
delete p;
cout << *p;    // UNDEFINED BEHAVIOR!
```

Fix: Set `p = nullptr;` immediately after delete.

[?] Spot the Bug

Bug 3: Integer overflow

```
int a = 2000000000;
int b = a + a;     // OVERFLOW! int max is ~2.1 billion
cout << b;         // prints negative garbage
```

Fix: Use `long long b = (long long)a + a;`

[ ] Refactoring Exercise

Bad Code — Magic numbers:

```
if (age > 18 && score > 75)
    discount = price * 0.15;
```

Good Code:

```
constexpr int    ADULT_AGE     = 18;
constexpr int    MIN_SCORE     = 75;
```

```
constexpr double DISCOUNT_RATE = 0.15;

if (age > ADULT_AGE && score > MIN_SCORE)
    discount = price * DISCOUNT_RATE;
```

13.2 Compiler Flags Reference

Table 13.1: Important GCC/Clang Compiler Flags

Flag	Purpose
`-std=c++17`	Use C++17 standard
`-Wall`	Enable all common warnings
`-Wextra`	Extra warnings
`-g`	Debug symbols (gdb/lldb)
`-O2`	Optimize for speed (release)
`-O0`	No optimization (debug)
`-fsanitize=address`	Detect memory errors at runtime
`-fsanitize=undefined`	Detect undefined behavior

Chapter 14

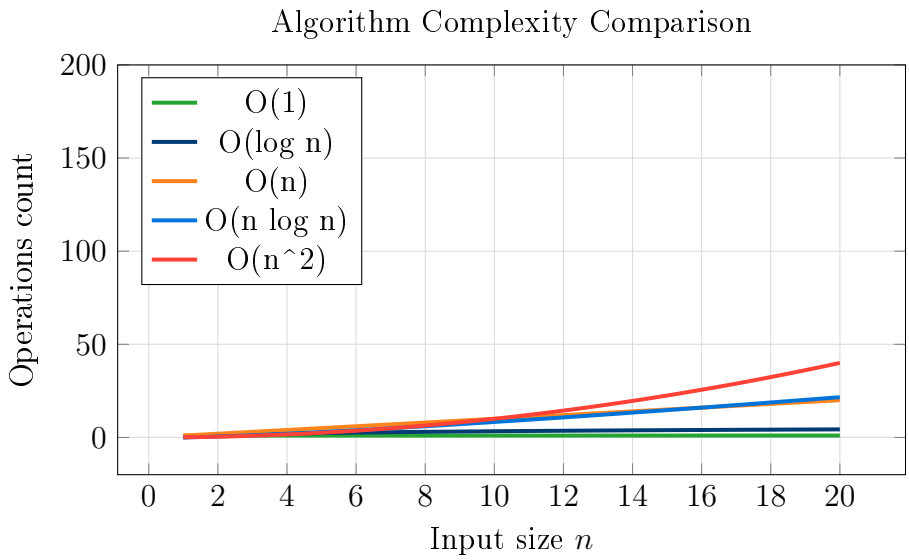
Performance & Big-O Analysis

14.1 Container Complexity

Table 14.1: STL Container Time Complexity

Container	Access	Insert	Delete	Search
`vector`	O(1)	O(1) amort.	O(n)	O(n)
`list`	O(n)	O(1)	O(1)	O(n)
`map`	O(log n)	O(log n)	O(log n)	O(log n)
`unordered_map`	O(1) avg	O(1) avg	O(1) avg	O(1) avg
`set`	—	O(log n)	O(log n)	O(log n)
`stack/queue`	O(1)	O(1)	O(1)	—
`priority_queue`	O(1) top	O(log n)	O(log n)	—

14.2 Algorithm Complexity Growth Chart



Appendix A
Quick Reference Card

Table A.1: C++ Cheat Sheet

Topic	Syntax
Variable	`int x = 5; auto y = 3.14;`
Const	`const int N = 100; constexpr int M = 200;`
Array	`int a[5] = {1,2,3,4,5};`
Vector	`vector<int> v = {1,2,3}; v.push_back(4);`
For loop	`for (int i=0; i<n; i++) { }`
Range-for	`for (auto& x : vec) { }`
Lambda	`auto f = [](int x){ return x*2; };`
Class	`class Foo { public: int x; void fn(); };`
Pointer	`int* p = &x; *p = 5;`
Reference	`int& r = x; r = 5;`
unique_ptr	`auto p = make_unique<int>(42);`
shared_ptr	`auto p = make_shared<MyClass>();`
Map	`map<string,int> m; m["key"]=1;`
Sort	`sort(v.begin(), v.end());`
Find	`auto it = find(v.begin(),v.end(),5);`
Template	`template<typename T> T f(T x) {...}`
Exception	`try {...} catch(exception& e) {...}`

A.1 Online Resources

- **cppreference.com** — Complete C++ API reference: https://cppreference.com
- **isocpp.org** — Official C++ website: https://isocpp.org
- **C++ Core Guidelines** — Best practices by Bjarne Stroustrup: https://github.com/isocpp/CppCoreGuidelines
- **TheAlgorithms** — Algorithms implemented in C++: https://github.com/TheAlgorithms/C-Plus-Plus
- **Godbolt Compiler Explorer** — Try code online: https://godbolt.org
- **Catch2 Testing Framework**: https://github.com/catchorg/Catch2